

Introduction to Python for Engineers

Agenda

- [Where do we start?](#)
- [An Example of a Python Program](#)
- [Background and History of Python Programming Language](#)
- [About this Course](#)
- [Lesson 1: Introduction to Jupyter Notebook](#)

Where do we start?

Let's do a short poll to see what you already know about Python.

1. I have never used Python before.
2. I have used Python before, but I am not very familiar with it.
3. I have used Python before and I am somewhat familiar with it.
4. I am a Python expert.

An Example of a Python Program

Below you see an example of a Python program. If you are used to other programming languages, this looks a bit different... but you'll get used to it!

```
In [3]: print('Python is fun!')
```

Python is fun!

```
In [4]: for g in range(3, 0, -1):  
        print(g)
```

3
2
1

As you can see, Python is easy to read, almost like English. You will learn more about the Python syntax later on.

Background and History of Python Programming Language

Why Python?

Python is a very popular programming language for a variety of reasons

- **Readability and Simplicity:** Python has a clean and readable syntax, which makes it easy to learn and understand.
- **Versatility:** Python is a general-purpose language, meaning it can be used for a wide range of applications, from web development to data analysis, machine learning, automation, and more. (e.g. our PLM system "CIM Database" embeds Python for customization)
- **Large Standard Library:** Python comes with a vast standard library that provides modules and functions for many common tasks, reducing the need for external libraries.
- **Extensive Ecosystem:** There is a rich ecosystem of third-party libraries and frameworks that extend Python's capabilities across different domains.
- **Community Support:** Python has a large and active community, which means extensive documentation, a wealth of tutorials, and community-contributed packages are readily available.
- **Cross-Platform Compatibility:** It can run on various operating systems like Windows, macOS, and Linux without modification.
- **Data Science and Machine Learning:** Python has become the go-to language for data science and machine learning, largely due to libraries like NumPy, Pandas, Scikit-learn, and TensorFlow.

Background and History of Python Programming Language

Let's trace the roots of this programming language

- <https://www.youtube.com/watch?v=Tm5u97I7OrM>
- https://www.youtube.com/watch?v=ucD_1ryKKm0

- <https://www.youtube.com/watch?v=J0Aq44Pze-w>

Different Ways to use Python

- **Python in "Jupyter Notebooks"**: This slide you see is a document, called Jupyter Notebook. It combines text / documentation with coding. In every code cell we can execute some Python. All cells know of each other. This is a very convenient way to do data analytics, since you can explain your thoughts, run your code, and visualize the results in the same place.
- **Standalone Python Applications**: You write your code in an IDE such as VS Code and execute it. There are many different types of applications, such as web applications, command line applications, or applications with a user interface that runs on your operating system.
- **Micro-Python**: This is a subset of Python to write code that runs on micro-controllers. This is becoming more and more popular for prototyping IoT applications. However, in terms of speed, it cannot compete with C/C++.
- **Python Scripting**: And many software systems use Python as a scripting language to enhance their functionality (like our PLM solution at OST, or AWS Lambda functions)

About this Course

Some Comments Upfront

1. The goal of this course is a dedicated introduction to the programming language for engineers
 - Basic usage of the language
 - Crunching data and connecting from different sources
 - Dealing with math and visualization
 - **New: Learn how to properly use AI for coding**
2. With 2 ECTS we can only scratch the surface
 - We will give you a lot of hints where you can do a deep dive yourself
 - If you want to become a programmer: you need to program, A LOT!
 - We highly recommend giving yourself challenges (AI is a good source to find ideas on how you can challenge yourself)
3. Language: The lectures and exercises are in English
 - Most of the documentation and examples you find are in English anyway
 - We plan to offer this course to incoming exchange students
4. There are tons of materials to read, learn, and get support online
 - Most of the time this course refers to: <https://docs.python.org/3/> and <https://www.w3schools.com/>
 - The first is comprehensive and accurate to the latest standard
 - The second one is simple, sets focus on the basics, and it looks the same for many programming languages

Content of this Course

1. The basics (Week 1-4)
 - Working with Jupyter Notebooks and VS Code
 - Python syntax, data types, and control structures, loops
 - Functions, namespaces, and libraries
2. Working with data (Week 5-7)
 - Extended list types: dictionary, tuples, sets
 - Crunching strings
 - Reading and writing files
3. Dedicated libraries for engineers (Week 8-10)
 - Pandas
 - NumPy
 - Matplotlib
4. Advanced programming skills (Week 11-14)
 - Using webservices
 - Vibe Coding

You will find all required links on Moodle. And all lecture notebooks can be accessed here:

<https://gitlab.ost.ch/edu-public/python-for-engineers/pyeng-lectures>

About coding and AI in this course

- AI, co-pilots, and vibe coding are great tools. **In fact, they changed the way I think about code!**
- We use them a lot for programming, they make us tremendously fast
- But these days, we cannot fully trust AI.

- Sometimes it does not understand the problem you are trying to solve,
- Sometime it hallucinates,
- It is not very transparent how it draws decisions
- AI has become a problem in cyber security, as it does not understand how the code is deployed or used after writing
- Vibe coding can be expensive (and use a lot of energy)
- Thus, AI is only a helpful tool if you can understand and control its output and the environment it is being deployed in.
- When using AI, make sure you know what you are doing, and where the code is being changed and run.

If you are new to a programming language, don't use co-pilots. You need to get familiar with the syntax and you need to figure out how to use the language yourself

Remark: this video explains very well what you should consider when learning Python, and how to use/not use AI/co-pilots:
<https://www.youtube.com/watch?v=St48epdRDZw>

Our Development Environment

- In this course we use **Visual Studio Code** as our IDE (most of you should be familiar with this)
- Most of the exercises will be done in a **Jupyter Notebook**
- All templates and exercises are delivered through **Moodle**
- We will push you to use **git** in a personal ***git repository***!
 - This is the way programmers work, and you won't lose your code
 - If you want support, this is the way to share code with us (Python files via email will be deleted immediately)
 - If you do vibe coding, there is now way around proper code management
 - For those who don't know git, go here: <https://www.youtube.com/watch?v=r8jQ9hVA2qs>
- In class we are working with **Python version 3.12**
- Feel free to use your own device and upgrade to version **3.14**. However, we will not support or assist with private installations.
- Lecture and exercises are also available on **gitlab**
 - <https://gitlab.ost.ch/edu-public/python-for-engineers/pyeng-lectures>
 - <https://gitlab.ost.ch/edu-public/python-for-engineers/pyeng-exercises>

Lesson 1: Introduction to Jupyter Notebook

About Jupyter Notebook

- As mentioned before, Jupyter Notebooks are a great way to program, explain, document and present results all in one place.
- Jupyter Notebook is a fusion of two very fundamental technologies
 - Markdown: A simple notation convention to create formatted text using a plain text editor
 - Python: Our programming language
- In short, a Jupyter Notebook is a simple file with the ending .ipynb
- Visual Studio Code (or JupyterLab, which is another tool to work with Jupyter Notebooks) helps you make these files interactive
- Inside these files we can create blocks
 - Markdown blocks to write text
 - Python blocks to write code
- Each Python block is followed by an output block where we see the results of the Python code.

Let's Create our First Notebook



You see, it's simple...

Here's What a Real Document Could Look Like

[Example](#)

Need more hints?

- This is a good introduction to jupyter notebooks: <https://www.dataquest.io/blog/jupyter-notebook-tutorial/>
- This is a good guide to work with jupyter in VSCode: <https://code.visualstudio.com/docs/datascience/jupyter-notebooks>

The most Important Markdown Rules

Sign	Explanation	Example
# Title	Heading 1	Heading 1
## Title	Heading 2	Heading 2
some text	Italic	<i>This is italic text</i>
some text	Bold	This is bold text
* Point	Bullet point	• Point 1
1. Item	Numbered item (always use 1.)	1. Item 1 2. Item 2
---	Horizontal line	
`code`	Code (monospace)	<code>code example</code>

With a fence of three backticks followed by `python`, you can format Markdown text as Python code:

```
``` python
print("Hello World")
```
```

which renders as:

```
print("Hello World")
```

For further reference go here: [Markdown Cheat Sheet](#)

We did it, that's all for today.

Now let's get our hands dirty!